Category B Assignment

PROBLEM 1 • Basic

Morning Wake-Up Circuit

A smart home app wants any device that can 'ring' — an alarm clock or a doorbell — to plug into one shared ringing system, even though the two devices have nothing else in common.

Requirements

- Define an interface Ringable with a method ring() returning a String.
- AlarmClock and Doorbell must both implement Ringable directly, with no shared parent class.
- Write a static method ringAll(Ringable[] devices) that loops through a mixed array and calls ring() on each.

Function Signature(s)

```
public interface Ringable { String ring(); }
public AlarmClock(String time)
public Doorbell(String location)
static void ringAll(Ringable[] devices)
```

Examples

Input	Output
`AlarmClock a = new AlarmClock("7:00 AM");` `a.ring()`	`"Alarm ringing for 7:00 AM"`

Input	Output
`Doorbell d = new Doorbell("Front Door");` `d.ring()`	`"Doorbell ringing at Front Door"`

Input	Output
`ringAll(new Ringable[]{ a, d })`	`"Alarm ringing for 7:00 AM"` `"Doorbell ringing at Front Door"`

HINT Ringable is the only thing AlarmClock and Doorbell need to share — resist giving them a common parent class they don't otherwise need.

CONSTRAINTS: time and location are never blank.

PROBLEM 2 • Basic

Gallery Description Cards

An art gallery app prints a description card for every piece, but a painting and a sculpture describe themselves completely differently — there's no such thing as a generic, undefined 'art piece' with no actual form.

Requirements

- ArtPiece must be an abstract class with an abstract method describe(), and a final pieceId field assigned via a shared static counter.
- Painting and Sculpture must both extend ArtPiece directly and implement describe() with genuinely different messages.

Function Signature(s)

```
public abstract class ArtPiece  // assigns a final pieceId internally
public abstract String describe();
String getPieceId()
public Painting(String title)
public Sculpture(String title)
```

Examples

Input	Output
`Painting p = new Painting("Sunset Fields");` `p.describe()`	`"Painting: Sunset Fields, framed on canvas"`

Input	Output
`Sculpture s = new Sculpture("The Thinker II");` `s.describe()`	`"Sculpture: The Thinker II, carved from stone"`

HINT The static counter belongs on ArtPiece itself, incremented once inside its own constructor — every subclass reaches it automatically through super().

CONSTRAINTS: title is never blank.

PROBLEM 3 • Intermediate

Backyard Toolshed Routine

A gardening app models tools as a family: every GardenTool can be used, and a Pruner is a cutting tool with its own extra trimming step layered on top of the general cutting-tool routine.

Requirements

- Pruner must extend CuttingTool, and CuttingTool must extend GardenTool — a genuine multilevel hierarchy three classes deep, with each subclass constructor calling its parent's constructor using super(...).
- GardenTool must be abstract with an abstract use() method; CuttingTool must override use() with its own concrete version; Pruner must override use() again.
- Each override (in CuttingTool and Pruner) must call super.use() first and then add its own extra detail.

Function Signature(s)

```
public abstract class GardenTool()
public abstract String use();
public CuttingTool()  // overrides use(), calls super.use()
public Pruner()  // overrides use() again, calls super.use()
```

Examples

Input	Output
`CuttingTool c = new CuttingTool();` `c.use()`	`"Using the tool in the garden, blade sharpened first"`

Input	Output
`Pruner p = new Pruner();` `p.use()`	`"Using the tool in the garden, blade sharpened first, then trimming branches precisely"`

Explanation: *Pruner reuses CuttingTool's own message via super.use() and appends its own detail, the same reuse pattern working one level or two levels deep.*

HINT Each use() override should call super.use() first, then add only the new detail that level introduces.

CONSTRAINTS: No arguments are required to construct any of the three classes.

PROBLEM 4 • Intermediate

Digital Classroom Setup

A school app manages classroom devices, and separately, some devices need charging — sometimes for a specific number of minutes, sometimes just a general top-up.

Requirements

- ClassroomDevice must be an abstract class with an abstract method operate().
- Define an interface Chargeable with a method charge() and an overloaded charge(int minutes) — true compile-time polymorphism.
- Tablet must extend ClassroomDevice and implement Chargeable, providing both charge() methods.

Function Signature(s)

```
public abstract class ClassroomDevice()
public abstract String operate();
public interface Chargeable { String charge(); String charge(int minutes); }
public Tablet(String assetTag)
```

Examples

Input	Output
`Tablet t = new Tablet("TAB-5");` `t.operate()`	`"Tablet TAB-5 displaying lesson"`

Input	Output
`t.charge()`	`"TAB-5 charging"`

Input	Output
`t.charge(30)`	`"TAB-5 charging for 30 minutes"`

Explanation: *charge() and charge(int minutes) are two separate methods sharing a name — Java decides which one runs based on how many arguments you pass in.*

HINT Let charge(int minutes) build its own message — it doesn't need to call charge() internally, though it's fine if that's simpler for you.

CONSTRAINTS: assetTag is never blank.

PROBLEM 5 • Intermediate

Skyline Delivery Fleet

A drone delivery company flies delivery drones that can be tracked and scout drones that can't be tracked at all, plus ground robots that can be tracked but aren't drones in any way.

Requirements

- Drone must be an abstract class with an abstract method fly().
- Define an interface Trackable with a method getLocation().
- DeliveryDrone must extend Drone and implement Trackable; ScoutDrone must also extend Drone (hierarchical — a sibling of DeliveryDrone) but must NOT implement Trackable.
- GroundRobot must implement only Trackable, with no relationship to Drone at all.
- Write a method that accepts any Object, checks with instanceof whether it implements Trackable, and if so, safely casts it to call getLocation() — otherwise, report that tracking isn't available.

Function Signature(s)

```
public abstract class Drone()
public abstract String fly();
public interface Trackable { String getLocation(); }
public DeliveryDrone(String id)
public ScoutDrone(String id)
public GroundRobot(String id)
static String getLocationIfTrackable(Object o)
```

Examples

Input	Output
`DeliveryDrone d = new DeliveryDrone("DR-1");` `getLocationIfTrackable(d)`	`"DR-1 at Sector 4"`

Input	Output
`ScoutDrone s = new ScoutDrone("SC-1");` `getLocationIfTrackable(s)`	`"Tracking not available"`

Input	Output
`GroundRobot g = new GroundRobot("GR-1");` `getLocationIfTrackable(g)`	`"GR-1 at Sector 4"`

Explanation: *GroundRobot has no relationship to Drone at all, but still works with the same instanceof check because it's Trackable that matters, not ancestry.*

HINT ScoutDrone sharing a parent with DeliveryDrone doesn't force it to share every capability — choosing not to implement Trackable is a perfectly valid design.

CONSTRAINTS: id is never blank.